

Markdown 渲染 PDF 指南（实际执行流程详解）

本文档结合本项目实际运作方式（Windows 本地 + 远程 Ubuntu 服务器），详细讲解把 Markdown 实验报告渲染为 PDF 的每一步。所有命令均在 **Windows 本地** 执行。

一、整体流程概览

```
graph TD
    A[实验报告.md (ReportSource/)] -- "1. md2pdf 命令行转换" --> B[实验报告.pdf (Report/)]
    B -- "2. git 提交" --> C[完成该题/该次练习的提交]
```

要点：**md 源文件与 PDF 分离存放**（源在 `ReportSource/`，成品在 `Report/`），每题/每次练习转换一次并随 git 提交。

二、使用的工具

- **工具**： `md-to-pdf-cli` (Python)，命令名 `md2pdf`
- **底层渲染**： `md-to-pdf-cli` 内部依赖 Playwright 启动 Chromium，把 Markdown 渲染成网页再打印为 PDF
- **安装方式**： `uv tool install md-to-pdf-cli`（或 `pip install md-to-pdf-cli`），首次运行自动下载 Chromium

三、详细执行步骤

步骤1：确认本地工具可用

```
md2pdf --help
```

若能列出子命令（`convert` 等）说明就绪。

步骤2：执行转换命令

项目中每次渲染实验报告都用同一条命令，**关键参数是** `--font "Microsoft YaHei"`：

```
# 转换到指定 PDF 路径（本项目必须 -o，因为目标目录/文件名不同）
set PYTHONIOENCODING=utf-8
E:\Python\Scripts\md2pdf.exe convert "ReportSource\实验报告_第6题.md" \
    -o "Report\实验报告_第6题.pdf" \
    --font "Microsoft YaHei"
```

逐项解释：

- `set PYTHONIOENCODING=utf-8`：把控制台编码设为 UTF-8。不加的话，Windows 默认 GBK 控制台打印 ✓ 报错码时会报 `UnicodeEncodeError: 'gbk' codec`（只是显示问题，PDF 仍能生成，但统一加上更干净）。
- `E:\Python\Scripts\md2pdf.exe`：`md2pdf` 的可执行文件所在完整路径（若已加入 PATH 可写 `md2pdf`）。
- `convert`：子命令，表示执行 Markdown→PDF 转换。
- 两个路径都要用双引号包（含中文文件名）。
- `--font "Microsoft YaHei"`：**指定中文字体，防止乱码**（详见第五节，本项目必填）。

步骤3：确认转换结果

转换成功会输出一行含 ✓ 的提示，随后在 `Report\实验报告_第6题.pdf` 生成 PDF 文件。

```
dir "Report\实验报告_第6题.pdf"
```

能列出文件即成功。随后可用阅读器打开检查中文是否正常、排版是否完整（表格、代码块、标题层级）。

步骤4：随 git 一并提交

有问题则改 md 后重转；没问题则把 **md 源 + PDF** 一起加入本阶段的提交：

```
git add -f "ReportSource\实验报告_第6题.md" "Report\实验报告_第6题.pdf" "Scripts/q06/"
git commit -m "完成第6题"
```

`-f` 用于确保加入（报告文件名含中文，规范一致则不强求）。每次完成一道题或一次课后练习都要提交一次。

四、批量/多文件转换

若一次完成多份报告（如课后练习同时交付），可循环处理：

```
set PYTHONIOENCODING=utf-8
mkdir -p Report
for /f "tokens=*" %f in ('dir /b ReportSource\*.md') do (
    md2pdf convert "ReportSource\%f" -o "Report\%~nf.pdf" --font "Microsoft YaHei"
)
```

在 cmd 批处理中 `%~nf` 取不含扩展名的文件名。也可先用 `md2pdf init` 生成 `md2pdf.toml`，在 `[theme].font_family` 写入 `"Microsoft YaHei"`，之后省略 `--font`。

五、中文乱码问题（本项目最关键的坑）

现象

默认或多数教程推荐的字体名 `Noto Sans CJK SC` 在 Windows **并没有安装**。Chromium 找不到该字体时会退化为 Type3 点阵字体，导致 PDF 中中文变成黑色方块 / 乱码。

判断方法

- 打开 PDF「文件→属性→字体」，若出现大量 `/Subtype /Type3`，即为乱码 fallback；
- 正确内嵌则应看到 `/Subtype /CIDFontType2` 且带 `/FontFile2` 的中文字体。

根治办法

显式指定本机**实际存在**、且能被 Chromium 正确内嵌的字体全名，推荐 `Microsoft YaHei`：

```
md2pdf convert 输入.md -o 输出.pdf --font "Microsoft YaHei"
```

备选中文字体（Windows）：`DengXian`（等线）、`SimSun`、`SimHei`。若在 Linux 服务器上做转换，需先 `sudo apt install fonts-noto-cjk` 再 `--font "Noto Sans CJK SC"`。

六、本项目实际使用结论

1. 每份实验报告都写在 `ReportSource\实验报告_第X题.md`（或 `实验报告_课后练习X.md`）。
 2. 用 `md2pdf convert ... --font "Microsoft YaHei"` 渲染到 `Report\` 同名 PDF。
 3. 生成后随 md、脚本一起 `git add` 并提交，完成一次交付。
 4. 全程在 Windows 本地执行（渲染不依赖远程服务器，远程只负责执行命令/脚本）。
-

参考

- 工具： `md-to-pdf-cli`（<https://github.com/Suyw-0123/md-to-pdf-cli>）
- 底层：Playwright + Chromium 打印渲染